

Virtualization

Laboratórios de Sistemas e Serviços

Mário Antunes

Universidade de Aveiro

2026

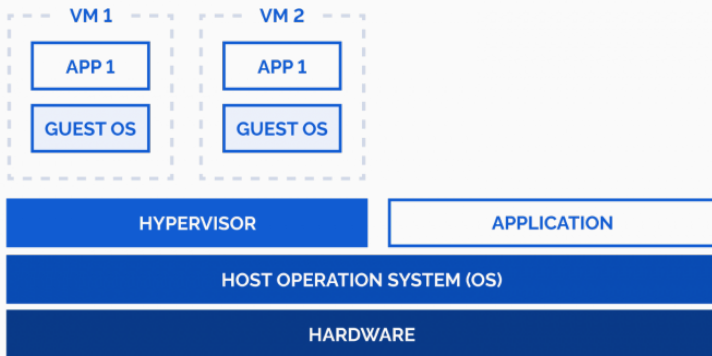
Introduction

What is Virtualization? i

Virtualization creates a software-based, or “virtual,” version of a computer. This Virtual Machine (VM) runs as an application on your physical computer but behaves like a completely separate machine.

- **Host:** Your physical machine and its Operating System (OS).
- **Guest:** The virtual machine and the OS it runs.
- **Hypervisor:** The software that creates and manages the VMs.

What is Virtualization? ii



The Challenge: Privileged Instructions

A normal application cannot access hardware directly; it must ask the Host OS. But a Guest OS **expects** to have full control. How do we solve this conflict safely?

The hypervisor's main job is to intercept and safely manage the guest's requests for privileged hardware access. The way it does this defines the difference between emulation and virtualization.

Virtualization Types

Definition: Emulation involves using software to mimic the hardware of a *different* system. The hypervisor acts as a translator, converting instructions from the guest's CPU architecture to the host's CPU architecture.

Use Case: Running a classic video game designed for an ARM-based console (like the Nintendo Switch) or a PowerPC-based console (like the GameCube) on your x86-based PC. The emulator (e.g., Yuzu or Dolphin) translates the game's code in real-time.

Emulation: Path of an Instruction

The hypervisor (emulator) must inspect and translate every instruction in software before it can be executed by the host hardware.

Emulation: Advantages & Disadvantages

Advantages

- **Cross-Architecture Compatibility:** Its greatest strength. It allows software designed for one type of CPU (e.g., ARM) to run on a completely different type (e.g., x86).

Disadvantages

- **Very Slow:** The software translation step for every instruction creates significant performance overhead, making it much slower than running native code.
- **High Resource Usage:** The translation process itself is computationally expensive and consumes a lot of host CPU cycles.

Full Virtualization: Definition & Use Case

Definition: Full Virtualization runs an *unmodified* guest OS on a simulated hardware environment that matches the host's architecture. It relies on **CPU hardware assistance** (Intel VT-x / AMD-V) to run code efficiently. The guest OS is completely unaware that it is being virtualized.

Use Case: A macOS user running a full version of Windows 11 in VirtualBox to use a specific piece of software that is not available on macOS, like a CAD program or a particular PC game.

Full Virtualization: Path of an Instruction

Non-privileged instructions run directly on the host CPU at full speed. When the guest attempts a privileged instruction, the CPU hardware automatically **traps** it and transparently hands control over to the hypervisor to handle it safely.

Full Virtualization: Advantages & Disadvantages

Advantages

- **High Compatibility:** Can run any standard, off-the-shelf operating system without any changes.
- **Good Performance:** Hardware assistance makes it significantly faster than emulation.
- **Strong Isolation:** Guests are securely isolated from the host and each other by the hardware.

Disadvantages

- **Trap Overhead:** The “trap-and-emulate” cycle for privileged instructions still introduces some performance overhead, which can be significant for I/O-heavy workloads.

Paravirtualization: Definition & Use Case

Definition: In Paravirtualization, the guest OS is *aware* that it is a VM and has been modified with special drivers. Instead of performing actions that would be trapped, it communicates directly with the hypervisor through an efficient API.

Use Case: This is the foundation of modern cloud computing. A high-performance web server running on an Amazon Web Services (AWS) EC2 instance uses paravirtualized **VirtIO** drivers for its disk and network devices to achieve maximum throughput and low latency.

Paravirtualization: Path of an Instruction

The guest OS knows it cannot directly access hardware, so its aware driver makes a **“Hypercall”** — a direct and highly efficient function call to the hypervisor, completely avoiding the trap mechanism.

Paravirtualization: Advantages & Disadvantages

Advantages

- **Highest Performance:** By avoiding the trap overhead, it offers the best performance, especially for disk and network-intensive tasks.
- **Efficient:** Lower CPU overhead compared to full virtualization because the communication path is optimized.

Disadvantages

- **Requires Guest OS Modification:** Cannot run an unmodified, off-the-shelf OS. The OS must have the specific paravirtualization drivers installed (though most modern OSes now include them).

Comparison Summary

Feature	Emulation	Full Virtualization	Paravirtualization
Core Concept	Mimic different hardware	Isolate an unmodified OS	Cooperate with an aware OS
Performance	Very Low	Good	Excellent
Guest OS Mod.	No	No	Yes
Hardware	Any guest on any host	Same architecture	Same architecture
Mechanism	Software Translation	Hardware Trap & Emulate	Hypercalls
Use Case	Retro Gaming, Cross-Arch Dev	Desktop, Legacy Systems	Cloud, Data Centers

Use Cases

Virtualization is the backbone of the modern cloud.

- **Server Consolidation:** One powerful physical server can replace dozens of older ones by running each as a separate VM, saving power, cooling, and space.
- **Snapshots & High Availability:** Instantly save and restore the state of a VM. VMs can even be migrated between physical servers with zero downtime.

The Problem: Repetitive VM Setup

Imagine you need to deploy 10 identical web server VMs. The manual process for *each one* would be:

1. Boot the VM and log in.
2. Set a unique hostname.
3. Configure the network.
4. Create user accounts and set up SSH keys.
5. Run security updates (`apt update && apt upgrade`).
6. Install necessary software (nginx, ufw, etc.).
7. Configure services.

This is slow, tedious, and prone to human error. It simply does not scale for cloud environments.

The Solution: Cloud-Init

Cloud-Init is the industry-standard tool for automating the **initial setup** of a cloud instance or virtual machine. It is designed to run **only on the very first boot** to provision the system.

- **How it Works:**

1. The cloud platform or hypervisor provides configuration data (called “user data”) to the VM as it is being created.
2. Inside the guest OS, a Cloud-Init service starts automatically on the first boot.
3. This service finds the user data and executes the instructions within it to configure the system.

- **Analogy:** Think of Cloud-Init as an automated setup script that configures your new server for you before you ever log in for the first time.

The configuration for Cloud-Init is typically written in **YAML**. This file, often named `user-data`, contains a set of directives. With this single file, a new VM can boot up fully configured with no manual intervention.

Cloud-Init in Practice: User Data ii

```
#cloud-config
hostname: webserver-01

users:
  - name: admin
    sudo: ALL=(ALL) NOPASSWD:ALL
    groups: sudo
    shell: /bin/bash
    ssh_authorized_keys:
      - ssh-rsa AAAA... user@example.com

packages:
  - nginx
  - ufw

runcmd:
  - [ ufw, allow, 'WWW Full' ]
  - [ systemctl, enable, --now, nginx ]
```

Virtual I/O and Networking

The I/O Challenge and VirtIO

A VM has no physical hardware. The hypervisor must provide virtual devices.

- **Emulated Devices (Slow):** The hypervisor pretends to be a real piece of hardware (like an Intel E1000 network card). Maximum compatibility, but slow.
- **Paravirtualized Devices (Fast):** Modern systems use **VirtIO**. The guest OS has a special virtio driver that uses a highly efficient, standardized channel to communicate with the hypervisor for disk and network tasks.

VirtualBox supports both: emulated devices for maximum compatibility, and VirtIO (paravirtualized) for performance when the guest OS has the drivers.

Virtual Networking: NAT vs. Bridged

- **NAT Mode (Default):** The VM shares your host's IP address. Easy to set up and allows the guest to access the internet, but makes it difficult for other devices on your network to connect to the guest.
- **Bridged Mode:** The VM gets its own unique IP address on your local network, appearing as a separate physical device. Ideal for running servers.

Device Passthrough: USB and PCI

You can grant a VM exclusive control over a physical device connected to your host.

- **USB Passthrough:** Gives a VM direct access to a USB device. Essential for embedded development, allowing your Debian VM to directly program an **Arduino or ESP32** board.
- **PCI Passthrough:** Assigns a physical PCI device, like a powerful **GPU**, directly to a VM. This offers near-native performance for demanding tasks like gaming or machine learning.

This advanced feature requires hardware support from the CPU and motherboard chipset, specifically the **IOMMU (Input-Output Memory Management Unit)**.

- **Intel's IOMMU:** VT-d
- **AMD's IOMMU:** AMD-Vi

The IOMMU creates a secure memory sandbox for the device, ensuring it can only access the memory of the VM it is assigned to. This prevents the device from interfering with the host OS or other VMs.

Oracle VirtualBox

Introducing VirtualBox

VirtualBox is a **Type-2 (hosted)** hypervisor that runs as a standard application on your existing OS. It is developed by Oracle and is free and open-source.

- **Who it is for:** Beginners, students, and desktop users who need an easy-to-use, graphical interface for running VMs.
- **Key Features:**
 - Cross-platform (Windows, macOS, Linux).
 - User-friendly graphical interface.
 - Guest Additions for seamless integration.
 - Easy-to-use snapshot functionality.
 - Support for VDI, VMDK, and VHD disk formats.

Installing VirtualBox

The process involves installing the main application and a separate Extension Pack for full functionality.

1. **Download:** Go to the [official VirtualBox downloads page](#) and download the package for your host OS. Also download the **Extension Pack** (adds USB 2.0/3.0, disk encryption, PXE boot).
2. **Install Application:** Run the installer for the main application.
3. **macOS Security:** On macOS, you must go to **System Settings > Privacy & Security** and **Allow** the system extension from Oracle.
4. **Install Extension Pack:** Double-click the downloaded .vbox-extpack file. VirtualBox will open and guide you through the installation.

Creating a VM in VirtualBox

Creating a VM is a straightforward, wizard-driven process.

1. Click the **"New"** button to start the new VM wizard.
2. Assign a **name** and select the **OS type** (e.g., "Debian 64-bit").
3. Assign **RAM** (e.g., 2048 MB) and **CPU cores** (e.g., 2).
4. When prompted for a hard disk, choose **"Do not add a virtual hard disk"** (for our class).
5. Go to **Settings > Storage** and click the disk icon to attach your provided .vdi file.
6. Select the VM and click **"Start"**.

VirtualBox: Guest Additions

Guest Additions are a set of drivers and utilities installed *inside* the guest OS to improve integration between host and guest.

- **Shared Clipboard:** Copy and paste text between host and guest.
- **Drag and Drop:** Drag files between host and guest windows.
- **Shared Folders:** Mount a host directory inside the guest for easy file exchange.
- **Auto-Resize Display:** The guest display automatically adjusts when you resize the VM window.
- **Seamless Mode:** Guest application windows appear directly on your host desktop.

To install: From the VM menu, select **Devices > Insert Guest Additions CD image**, then run the installer inside the guest.

VirtualBox: Snapshots

Snapshots capture the **entire state** of a VM at a specific point in time, including disk contents, RAM, and device configuration.

- **Why use them:**

- Save a known-good state before making risky changes.
- Instantly revert if something goes wrong.
- Create branching configurations from a common base.

- **How to use:**

1. Open **Machine > Take Snapshot** (or press Ctrl+Shift+S / Host+T).
2. Give the snapshot a descriptive name (e.g., "Before kernel update").
3. To restore: right-click the snapshot and choose **Restore**.

Tip: Snapshots grow in size over time. Delete old snapshots you no longer need to reclaim disk space.

VirtualBox: Networking Modes

VirtualBox offers several networking modes. The two most common are:

Mode	Guest to Internet	Host to Guest	Guest to Guest
NAT	Yes	Via Port Forwarding	No
Bridged	Yes	Yes (own IP)	Yes
NAT Network	Yes	Via Port Forwarding	Yes
Host-Only	No	Yes	Yes
Internal	No	No	Yes

For this class, we use **NAT** with port forwarding to access the VM via SSH.

VirtualBox: NAT Port Forwarding

In NAT mode, the guest is hidden behind the host's IP. **Port forwarding** lets you reach guest services from the host.

To set up SSH access to your VM:

1. Select your VM and open **Settings > Network > Adapter 1**.
2. Confirm the adapter is set to **NAT**.
3. Click **Advanced > Port Forwarding**.
4. Add a new rule:

Name	Protocol	Host IP	Host Port	Guest IP	Guest Port
SSH	TCP	127.0.0.1	2222	10.0.2.15	22

5. Connect from your host terminal:

```
$ ssh -p 2222 student@localhost
```

VirtualBox: Storage and Disk Formats

VirtualBox supports several virtual disk formats:

- **VDI (Virtual Disk Image):** VirtualBox's native format. Supports dynamic allocation and snapshots.
- **VMDK:** VMware's format. Useful for compatibility if you need to move VMs between VirtualBox and VMware.
- **VHD/VHDX:** Microsoft's format. Compatible with Hyper-V.

Dynamic vs. Fixed allocation:

- **Dynamically allocated:** The disk file starts small and grows as you add data, up to the maximum size. Saves host disk space.
- **Fixed size:** The full disk space is allocated immediately. Slightly faster I/O performance.

For this class, we use a pre-built `.vdi` file with dynamic allocation.

VirtualBox: VBoxManage CLI

For advanced users, VirtualBox provides the VBoxManage command-line tool. It exposes every feature available in the GUI and more.

Useful examples:

```
# List all registered VMs
$ VBoxManage list vms

# Start a VM in headless mode (no GUI window)
$ VBoxManage startvm "MyVM" --type headless

# Take a snapshot from the command line
$ VBoxManage snapshot "MyVM" take "clean-state"

# Set up a NAT port forwarding rule
$ VBoxManage modifyvm "MyVM" --natpf1 \
  "ssh,tcp,,2222,,22"
```

This is especially useful for scripting and automation.

QEMU + KVM

Introducing QEMU + KVM

QEMU is a powerful machine emulator, and KVM (Kernel-based Virtual Machine) is the Linux kernel's built-in virtualization module. Together, they provide high-performance, **Type-1 (bare-metal)** virtualization on Linux.

- **Who it is for:** System administrators, developers, and power users who need flexibility, performance, and command-line control. It is the engine behind many large-scale cloud platforms.
- **Key Features:**
 - Extremely flexible and scriptable.
 - Can emulate a huge variety of CPU architectures (ARM, MIPS, etc.).
 - Near-native performance when used with KVM.
 - Advanced storage with `.qcow2` (snapshots, thin provisioning).

Installing QEMU + KVM

On Debian/Ubuntu-based systems, installation is done via the apt package manager.

1. **Install Packages:**

```
bash $ sudo apt update $  
sudo apt install qemu-system-x86 \      kvm  
virt-manager libvirt-daemon-system
```
2. **Add User to Groups:** You will need to log out and back in for this to take effect.

```
bash $ sudo adduser  
$USER libvirt $ sudo adduser $USER kvm
```

The virt-manager package provides a graphical tool for managing QEMU/KVM VMs.

While `virt-manager` provides a GUI, the command line demonstrates QEMU's power.

1. **Create a Virtual Disk:** The `.qcow2` format is recommended. This creates a 20 GB disk that only grows as data is added.
`bash $ qemu-img create -f qcow2 \ my_debian_disk.qcow2 20G`
2. **Launch a VM from an ISO:** `bash $`
`qemu-system-x86_64 -enable-kvm \ -m 2048`
`-hda my_debian_disk.qcow2 \ -cdrom`
`debian-13-netinst.iso -boot d`
 - `-enable-kvm`: Use KVM for hardware acceleration.
 - `-m 2048`: Assign 2048 MB of RAM.
 - `-hda`: Primary hard disk file.
 - `-cdrom`: Attach an ISO as a virtual CD-ROM.
 - `-boot d`: Boot from the CD-ROM drive first.

Comparison and Resources

Comparison: VirtualBox vs. QEMU/KVM

Feature	VirtualBox	QEMU + KVM
Type	Type-2 (Hosted)	Type-1 (via Linux Kernel)
Platform	Windows, macOS, Linux	Linux
Ease of Use	Very High (GUI wizard)	Medium to Low (CLI)
Performance	Good (desktop use)	Excellent (near-native)
Flexibility	Good	Very High (scriptable)
Best For	Students, desktop users	Servers, cloud, developers

For this class, we use **VirtualBox** because it runs on all host operating systems and provides a simple graphical interface for managing VMs.

Bookmark these pages for quick reference.

- **VirtualBox:**
 - [User Manual](#)
 - [Networking Guide](#)
 - [Guest Additions](#)
- **QEMU:**
 - [User Manual](#)
 - [Networking](#)
- **Cloud-Init:**
 - [Documentation](#)